

Accessible Smart Contracts Verification: Synthesizing Formal Models with Tamed LLMs

Jan Corazza

Research Center Trustworthy Data Science and Security
TU Dortmund University
Dortmund, Germany
jan.corazza@tu-dortmund.de

Ivan Gavran

Informal Systems
Vienna, Austria
ivan@informal.systems

Gabriela Moreira

Informal Systems
Joinville, Brazil
gabriela@informal.systems

Danielneider

Research Center Trustworthy Data Science and Security
TU Dortmund University
Dortmund, Germany
daniel.neider@tu-dortmund.de

Abstract—When blockchain systems are said to be *trustless*, what this really means is that all the trust is put into software. Thus, there are strong incentives to ensure blockchain software is correct—vulnerabilities here cost millions and break businesses. One of the most powerful ways of establishing software correctness is by using formal methods. Approaches based on formal methods, however, induce a significant overhead in terms of time and expertise required to successfully employ them.

Our work addresses this critical disadvantage by automating the creation of a formal model—a mathematical abstraction of the software system—which is often a core task when employing formal methods. We perform model synthesis in three phases: we first transpile the code into model stubs; then we “fill in the blanks” using a large language model (LLM); finally, we iteratively repair the generated model, on both syntactical and semantical level.

In this way, we significantly reduce the amount of time necessary to create formal models and increase accessibility of valuable software verification methods that rely on them. The practical context of our work was reducing the time-to-value of using formal models for correctness audits of smart contracts.

Index Terms—Code Generation, Large Language Models, Formal Methods, Model Synthesis, Smart Contracts, Model-Based Techniques, Software Auditing

I. INTRODUCTION

Modern blockchain platforms such as Ethereum and Cosmos extend the underlying consensus technology pioneered by Bitcoin towards a generalized computing platform. Rather than solely reaching consensus on transactions modifying the state of a ledger, these platforms also enable consensus on program executions. Roughly speaking, they are platforms for decentralized computing capable of executing stateful protocols—called smart contracts—in a trustless manner. For example, one may write a smart contract for holding an auction without having to put one’s trust into a centralized marketplace provider such as eBay.

Currently, the smart contract ecosystem manages assets worth billions of dollars. This substantial value makes smart contracts prime targets for cyberattacks. A successful exploit

can result in catastrophic financial losses, potentially devastating businesses and the investments of numerous individuals.

Due to these high stakes, companies launching blockchain-based products have begun placing an increasingly high priority on carrying out thorough software audits before the launch. Most auditors, however, base their work on manual line-by-line code inspection, foregoing more powerful tools such as formal methods.

Formal methods are a potent tool in the software verification toolkit, particularly for high-stakes environments like blockchain-based systems. They rely on rigorous logical reasoning to either automatically uncover vulnerabilities or assert their absence, thereby mathematically guaranteeing that certain types of bugs do not exist. Not everything is perfect, however: for large codebases and expressive languages, formal methods scale poorly. One common way to address the problem of scalability is to build a formal model of the system and reason about that model instead of the full system implementation. We call the techniques embracing this approach *model-based techniques* (MBT).

A formal model is a mathematical abstraction of the software system—the art of modeling is about abstracting away all unnecessary details while preserving relevant aspects of the system. For example, in a banking application, a model will capture how funds change owners. When verifying the model, we can ensure that users cannot access funds that do not belong to them.

We can reason about properties of the model, and establish the connection of the model to the software using model-based testing [1] techniques. Thus, model-based techniques provide an effective framework for verifying software. However, because of the initial cost of writing a model and the need for specialized knowledge of the modeling language, model-based techniques did not yet see widespread adoption.

In this work, we reduce the effort for an auditor writing a model, with help of large language models (LLMs). We tame

LLMs’ propensity for hallucination by setting a hard frame around it through mechanical model synthesis, and through iterative repair of the generated model, until it conforms to the user’s (partial) specification.

An auditor’s workflow (model-based techniques): Let us examine the workflow of an auditor who attempts to use model-based techniques (we get into more details of those techniques in Section III-A). After understanding the logic of the codebase, the auditor **models** it in a modeling language. Now the auditor is able to reason about the model, but a connection to the implementation is missing. Thus, the auditor **writes some glue code** (called an *adapter*), which translates model’s abstract traces into test cases.

These two steps are already time-consuming for an auditor already experienced in MBT. For auditors just starting with MBT, they can also be daunting. Given that audits are typically time-constrained, the time that needs to be invested is often used as an argument against using (otherwise effective and useful) model-based techniques.

Therefore, the primary objective of our work is to significantly reduce the time and effort needed to apply model-based techniques to smart contracts. To this end, we propose an automated method that generates a formal model of a smart contract, based on the contract itself and the auditor’s¹ input.

Overview of the method: Our method allows the auditor-user to automate parts of the MBT workflow, or take over in the middle of the process. Importantly, at every step, the method provides a valuable artifact to the auditor.

As a first step, we transpile the smart contract(s) into a stub of the model: the stub captures interleavings of different actions, but does not capture the actions’ semantics. As a part of the transpilation process, we are also generating the stub of the adapter. At this point, the auditor can use both stubs to run fuzzing tests against the contract.

In the second step, we ask the user to provide a natural language (NL) description of each action and a couple of input-output examples. (An action is a transformation of the state, so the notion of I/O examples is well-defined.) Our method then interacts with an LLM: each time the generated model is not aligned with the user-provided examples, it generates the prompt pointing out the inconsistency and requesting new generation. Throughout our work we utilize OpenAI’s Chat Completions API for this purpose, specifically GPT-4o.

The main insight on which we based the design of our tool is that the careful separation of concerns between mechanical synthesis (transpilation) and statistical synthesis (LLM-based) is crucial. Relying on an LLM to generate the whole model results in hallucinations (e.g., using non-existent helper functions) or convincingly looking yet wrong models. Conversely, transpiling the full contract into a model is hopeless: even if it worked, it would result in line-by-line re-implementation of the contract, instead of a good abstraction. We suggest *taming*

¹We refer to an auditor as the main user of our method because our work was done in the context of a team performing correctness audits. The method is equally useful for developers aiming to employ formal methods in their work.

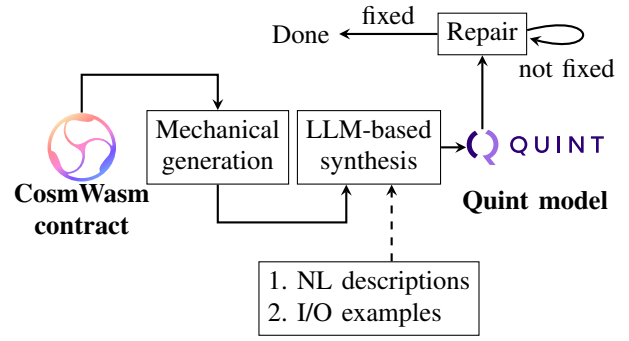


Fig. 1. High-level illustration of the method for generating a Quint model from a CosmWasm contract.

as an appropriate metaphor: model stubs give a firm frame that constraints the wild creative power of LLMs.

We focus on smart contracts implemented using CosmWasm [2], a smart-contract platform of the Cosmos ecosystem. CosmWasm contracts are written in Rust, and the models that our tool generates are written in Quint [3], an executable modeling language based on TLA—temporal logic of actions [4]. Figure 1 provides a high-level overview of our method for transforming a CosmWasm contract into a Quint model. The process involves three main steps.

- 1) **Mechanical Generation:** Transpile as much of the Rust source code into Quint, leaving placeholders (“stubs”) for parts that cannot be easily transpiled.
- 2) **LLM-based Synthesis:** Use a large language model (LLM) to complete the stubs based on user’s input, thereby finishing the Quint model.
- 3) **Testing and Repair:** Conduct a series of tests on the generated model and identify the LLM’s syntactic and semantic hallucinations. Then, provide the information about the problem to the LLM and ask for it to be fixed.

In Sections II and III, we discuss related work and provide the necessary background information, respectively. In Section IV, we elaborate the technical details involved in each step of our method.

To evaluate its effectiveness, we created a benchmark based on CosmWasm CTF [5], a collection of publicly available CosmWasm smart contracts with injected bugs. Along with the evaluation results on the CTF benchmark, we also conducted two case studies of using our method as a part of an auditor’s work. The evaluation results (described in Section V) suggest the following conclusion: our method successfully generates models for the benchmark contracts and is useful for auditors. Using the method, we uncovered a bug in a real-world smart contract.

II. RELATED WORK

In this section, we provide an overview of existing research and technologies related to formal methods for blockchain systems and LLM code generation, and place our work within this broader context.

A. Formal Methods for Blockchain Systems

Software vulnerabilities in the blockchain world imply high financial and reputational loss. Once on-chain, the software with a fix cannot be deployed quickly. Therefore, there has been a lot of effort in ensuring blockchain systems are correct.

On the level of the underlying blockchain infrastructure, engaging software verification experts to assess the correctness of the critical parts has become a standard. The experts employ a variety of tools to conduct their analysis, for instance: Jepsen [6] for fault tolerance, modeling with TLA+ [7] for proving properties of inter-blockchain communication [8], or providing a reference implementation [9] in Dafny [10].

Verifying individual smart contracts is (in principle, at least) a more attainable task than verifying the infrastructure, while the incentives remain equally high. Thus, there is abundance of tools for smart contracts correctness assurance. Examples include Certora [11], Slither [12], Mythril [13], Echidna [14], Quint [3], to name but a few.

One thing these tools all share is that only specialized experts can use them. The problem is well recognized and there is a trend for tools to get closer to users by integrating with commonly used frameworks. This approach is taken by e.g., Kontrol [15] and Halmos [16].

In this work, we take a different approach: we bring a modeling language closer to the users with help of automated synthesis. Such an approach could be applied to other formal verification tools as well.

B. LLM Code Generation and Verification

LLMs optimized for code generation showed stunning results [17], [18]. One concern when using LLMs for generating code is that they are inherently stochastic and offer no assurance that the generated code will be error-free.

Techniques such as PICARD [19] and Synchromesh [20] utilize parsers to sequentially filter out undesirable tokens at inference time. Synchromesh additionally introduces Target Similarity Tuning (TST), a mechanism for selecting few-shot demonstrations for in-context learning. While TST is highly relevant for all techniques which rely on few-shot prompting, we did not incorporate it in our approach due to the small number of few-shot demonstrations at our disposal and generally satisfactory performance without it. We discuss our prompting methodology and approach to LLM-based code synthesis in Section IV-B.

At their core, these methods enforce syntactic and a limited number of semantic rules by restricting the token set the LLM can sample from, based on a provided syntax. However, the rules enforceable at the parser level are limited and require an expert to encode them. Furthermore, while correct-by-construction synthesis is a valuable ideal, we found that code generation capabilities of advanced LLMs have progressed to a point where existing inference-time approaches, especially those that rely on syntax level guarantees, are becoming less relevant.

Techniques like Clover [21] cross-verify multiple LLM-generated artifacts, including code, documentation, and formal

specifications. This bears similarity to our approach, in which the LLM generator gets feedback from other generated parts (discussed in more detail in Section IV-C). A radically different approach is taken by Sun et al. [22], who experiment with changing a programming language's grammar to make it a better fit for LLM-based generation.

There has been a lot of interesting work with promising results combining aspects of formal methods and LLM code generation [23], [24], [25], [26], [27], [28]. Our work fits into that broad context.

III. PRELIMINARIES

In this section, we cover the necessary background on model-based techniques, the Quint modeling language, and CosmWasm smart contracts, which form the primary focus of our work.

A. Model-based Techniques

In the context of this paper, a *model* is an abstract, formal representation of a software system that can be automatically analyzed. Common examples include 1) writing models in the TLA+ [7] modeling language and analyzing them either by an explicit model checker TLC [29] or by a symbolic model checker Apalache [30]; and 2) writing models in the Alloy modeling language and analyzing them with the Alloy analyzer [31]. In this paper, we focus on models written in Quint (see Section III-B for a brief introduction to Quint).

Models are in general the most useful when modeling a (distributed) protocol *before* the implementation. Then, verifying the model can point to protocol problems early in the process (or assure us that the approach makes sense). In our auditing practice, we most often come in once the software is already implemented, and we focus on finding bugs. We can use models to help us find bugs in two ways: by verifying the *code-based* model, or by generating tests from the *ideal-based* model.

In the code-based approach, we write our model to be as close as possible to the implementation. Then, finding violations of invariants with respect to the model will correspond to finding bugs in the code. Of course, the pending question is how we can be sure that our model corresponds to the code: this is ensured by running model-based tests. The model generates traces, which are then executed against the implementation to verify conformance.

A symmetrical situation occurs in the ideal-based approach. There, we write a model to correspond to what we understand to be the correct functioning of the software. From the model, we generate traces which are executed against the implementation and after each step of the trace, the state of the model is compared to the state of the implementation. If there is a mismatch, it suggests there is a bug in the implementation. A similar validity question is pending here, too: how can we be sure that our model is correct? We can be assured of the model's correctness by writing and verifying the invariants that are expected to hold.

These two approaches represent somewhat extreme cases. In reality, a model, tests, and invariants may co-evolve as our understanding of the system improves. Furthermore, as noted already by many practitioners, one less tangible value of writing models is that it makes our thinking about the (audited) software much more precise. Thus, it is very common that bugs are recognized while writing the model, before any automatic analysis is run.

B. Quint

Quint [3] is a modeling language based on TLA+ [7]. Quint's main advantage compared to TLA+ is being significantly more in line with standard development practice.

It was designed according to the following principles:

- Whenever a language concept has a “standard” syntax in mainstream languages, Quint adopts it. Conversely, when a concept is specific to modeling languages, Quint's syntax makes it explicit. As examples: Quint expresses inequality with `!=` (as opposed to `#` and `/=`, used in TLA+); when a value needs to be chosen nondeterministically, Quint demands a `nondet` modifier.
- Users can specify types, and types are checked as soon as possible, identifying mistakes sooner rather than later.
- Indentation is encouraged with IDE tools, but does not carry meaning.
- There is a clear separation between functions that modify the model state, and those that do not.
- One can interact with parts of the model (i.e., in a REPL), before the whole model is finished.
- Models written in Quint remain compatible with TLA+ and transpilation is possible, so all existing tools for TLA+ can be used for Quint.

While most of these points may seem straightforward, the accepted wisdom among TLA+ users was that a modeling language does not need to follow modern programming practices. Even more, that trying to stay sufficiently away from programming languages' mores is beneficial for modeling [7].

In our experience, Quint's principles add value by integrating modeling into the development process, ergonomic tools, and intuitive language, and this has been essential for bringing modeling into practice.

C. CosmWasm Smart Contracts

CosmWasm [2] (CW) is a smart contract platform of the Cosmos ecosystem. The name comes from the fact that CW contracts are compiled into WebAssembly (Wasm), enabling CW contracts to be written in multiple languages (the de facto standard programming language for CW at this point is Rust).

A CosmWasm smart contract consists of type definitions which define how the contract's data is stored on the blockchain, and entry-point implementations that operate on that data based on the messages that the contract receives. Multiple contracts can communicate by sending each other messages, which are all delivered and processed as part of the same transaction as the initial user's message.

IV. MODEL GENERATION

As stated in the introduction, our goal is to generate a Quint model for a given smart contract in three steps. In the first step, which we call mechanical generation, we use the contract's Rust source code to determine the overall structure of Quint model. This enables us to generate data types, function signatures, and parts of code that follow common CosmWasm patterns, such as receiving and replying to messages and mutating the contract's internal state. However, we leave placeholders, which we refer to as stubs, in place of functions that encode the contract's main logic. Filling these placeholders mechanically would require one to develop a complete Rust-to-Quint transpiler, including translation of side effects and loops, which is an exceedingly difficult task. Additionally, mechanical generation would necessarily follow the Rust code by the letter, failing to sufficiently abstract away from it. (In Figure 3 in Appendix C of the extended version of this work [32], we provide an example of a stub along with the original Rust code that induced it.)

Instead, in the second step, we prompt the LLM to complete the missing parts of the model. In order to help the LLM accurately complete this task, we provide it with general information about Quint, user-provided NL descriptions, and I/O specifications. We also include relevant parts of the contract's Rust implementation along with results of step 1 (primarily, stubs' signatures that the synthesized code must adhere to). Most importantly, we provide the LLM with few-shot examples which demonstrate how to generate Quint code.

Finally, in the third step, we conduct a series of tests on the generated model to ensure its correctness, and again employ the LLM to repair any identified errors based on the error details.

A. Mechanical Generation for Stubs

The mechanical generation phase analyzes a CosmWasm contract written in Rust and produces a corresponding Quint model with the overall structure, without attempting to generally translate Rust expressions. It introduces placeholders for the core elements that it cannot translate. It also generates an adapter test file in Rust, which reads traces produced by the generated model and replays them against the implementation. The tool is called `cosmwasm-to-quint` and the source code is available on GitHub².

There are several ways to work with Rust code, from building a new parser to using existing translation libraries. After considering a few of those options, we chose a tool called `rustc_plugin` [33] for that purpose. `rustc_plugin` is a framework that makes it easier to add callbacks to compilation phases of `rustc` (the Rust compiler), where all the information produced by the compiler is available.

Since the compiler is capable of understanding the project's configuration (`Cargo.toml` and related files), the tool can be run for the entire project (as opposed to individual Rust files), which is a great convenience as CosmWasm projects

²<https://github.com/informalsystems/cosmwasm-to-quint>

often involve multiple files. Some projects define more than one contract, each one in its own Rust crate. The generation tool will generate a different Quint file and Rust test for each crate. A significant constraint that affects the transpilation at this time is the lack of import resolution for external crates. A potential solution for this is to download the external crates and compile them as well, so the definitions are compiled and compilation callbacks are called for them. This is not currently automated by the tool, which means any definitions from external crates will be undefined in the Quint model (and therefore, name resolution for Quint will report errors).

Finding the contract state: CosmWasm contracts have a state, and it has to be translated into a Quint state variable in order to be persisted between different transitions. Listings 1 and 2 show the input and the output of this process. The most complex part is finding which definitions are related to the state definition, and expressing their types (as Quint state variables require a precise type annotation). To this end, the tool scans all Rust constants and selects them as state variables if they satisfy all of the following conditions:

- 1) the body of the constant is a call to a type with a relative path that resolves into a definition;
- 2) the resolved definition is from a crate called “cw-storage-plus”;
- 3) the resolved definition is either “Map::new” or “Item::new”.

If “Map::new”, the tool looks for two generic arguments to determine the type of the keys and the values of the map, and produces a Quint type from them. If “Item::new”, the tool looks for a single generic argument, which is translated into the corresponding Quint type.

After collecting all state-related variable names and types, the generator will produce a Quint type called `ContractState`, defined as a record where field names are the variable names, and field types are their corresponding types. This is more convenient than generating one state variable for each of the found variables because it is necessary to pass this to many functions, so having the information stored in a single value makes it easier. The tool will always produce a state variable definition called `contract_state` with this type, and also a value definition called `init_contract_state` with the default values with which to initialize the contract state.

```
1 pub const LAST_ID: Item<u64> = Item::new("lock_id");
2 pub const LOCKUPS: Map<u64, Lockup> = Map::new("lockups");
```

Listing 1. Original CosmWasm state definition

```
1 type ContractState = {
2   last_id: int,
3   lockups: int -> Lockup
4 }
5 var contract_state: ContractState
```

Listing 2. Generated Quint code for the contract state

Finding actions: Quint actions define the transitions of the model, and they are the only kind of definition where

state variables can be modified. CosmWasm uses values of types `Deps` and `DepsMut` to pass the state around, the latter allowing for it to be updated. The tool looks for the presence of those values to determine if a Rust function should be translated into a Quint action or just a pure definition (which is the Quint version of a pure function).

When the tool generates a Quint action, it will also generate a matching pure definition that takes the two parameters in addition to the Rust parameters for the function: `state: ContractState` and `deps: Deps` parameter. In Quint, there is no distinction between `Deps` and `DepsMut` since there is no mutability. The pure definition will return the updated value of the contract state. Since there is no translation of the body of Rust functions, this will be a stub like the one in Figure 3 in Appendix C of the extended version of this work [32].

The generated action itself non-deterministically picks values for all of the Rust function’s parameters (except for `Deps`), builds an execution message with those parameters and calls `execute_message` with it. `execute_message` is a helper action that non-deterministically picks additional metadata (the sender address, funds denomination and funds amount) and calls the `execute` pure definition, which is a direct translation of the CosmWasm `execute` function, present in every contract. This is where each message will be processed by the pure definition generated for it (explained in the paragraph above). At each step of the model, `execute_message` is called, its result value is assigned to the contract state, and the bank is updated with the funds transfer according to the non-deterministic funds pick.

The bank is a separate state variable that stores the balances for each account. On top of the bank and the contract state, the model also has a variable to track time, and a variable to store the last result—the returned value from the last entry point that was executed which, like in CosmWasm, is either an error or a response object (containing arbitrary attributes and any number of messages).

Messages: Another core feature of CosmWasm’s execution environment is how messages are processed. A contract is not able to directly update the bank or call other contracts—it does that through messages. At this point, the mechanical generation tool is able to handle bank messages but not general inter-contract communication. That is an important improvement to be made.

Handling bank messages in the Quint model means that, at every step, if there is a message in the last execution’s response, the step should consist of processing that message instead of another execution call. It then updates the bank with the corresponding data inside the message and removes it from the set of messages in the response state variable.

The same concept can be applied for inter-contract communication, as that interaction is also done by including a message in the response. There is additional complexity in processing these messages because it involves other contract’s state and types.

Generated adapter: With the goal of bridging the gap between the Quint model and the CosmWasm code, the generator tool produces an adapter file. This is a Rust test that reads JSON traces produced by Quint tooling and reproduces each step of the trace in the CosmWasm contract, using the testing library `cw_multi_test` [34], which defines helpers to interact with the contract. If the trace can be reproduced successfully, and the states after each execution match, it is evidence that the set of behaviors defined by Quint model is also defined in the implementation. This test can be run for an arbitrary number of traces—the larger the number of traces we generate, the higher our confidence is that the implementation conforms to the model.

B. LLM-based Synthesis for Contract Logic

In this phase of the generation, we instruct the LLM to complete the Quint stubs, i.e., the partial model with placeholders in place of functions that implement the contract logic. In addition to the stubs that resulted from the mechanical generation phase, we assume that the following three types of inputs will be made available by the user.

- 1) CosmWasm source code, which contains the data structures that the contract stores on the blockchain, Rust implementations of message handlers, and helper data structures such as possible error formats.
- 2) Natural-language descriptions of the functionality that each stub is supposed to encode, which will steer the LLM’s output towards Quint code that correctly captures the contract logic.
- 3) Two input-output specifications for each placeholder, which will be used to steer generation, and repair the generated code in case of errors.

The result of the mechanical generation phase is a Quint model with “stubs” in place of implementations of pure Quint functions that capture the functionality of the smart contract message handlers. They correspond to CosmWasm message handlers in the contract’s Rust source code (and any utility functions that they may use). Stubs are *pure* Quint functions that are intended to capture the main contract logic. Pure functions are defined solely by their input-output behavior and have no side effects, i.e., they do not modify or read any external state implicitly, nor perform any I/O operations. The core idea of our approach is to factorize the complex task of generating an entire model into a set of more tractable pure function synthesis tasks. Crucially, the LLM may complete each stub *independently* of others. This also applies to testing and repairing the generated implementations.

Relying on this factorization, we provide a pseudocode outline for our LLM-based synthesis method in Algorithm 1. This pseudocode is run for each of the stubs. In Section IV-B (*In-Context Learning*) we explain how the LLM is prompted to generate the code for a particular stub. We explain the final phase of testing and repairing each generated implementation in Section IV-C. Throughout this work, we sample the LLM

using a temperature of 0.6 and a top-p value of 0.7. Our implementation of LLM-based synthesis is available on GitHub.³

Algorithm 1 LLM-Based Synthesis and Repair (Single Stub)

Require: *partialModel*, *cosmWasmSource*, *nlDescriptions*, *ioExamples*, *stubName*, *stubDescription*

- 1: *prompt* $\leftarrow$ GENERATEPROMPT(*stubName*, *stubDescription*, *ioExamples*, *cosmWasmSource*, *partialModel*)
 - 2: *code* $\leftarrow$ CALLLLM(*prompt*)
 - 3: *stub* $\leftarrow$ EXTRACT(*stubName*, *partialModel*)
 - 4: *filledStub* $\leftarrow$ REPLACE(*partialModel*, *stub*, *code*)
 - 5: **return** TESTANDREPAIR(*stubName*, *ioExamples*, *filledStub*)
-

In-Context Learning: It is well-understood that large language models are capable of in-context learning [35], i.e., learning from examples prepended to the main prompt. Instead of fine-tuning the LLM on a downstream task such as code generation, which is a slow and data-hungry process, one may improve the model’s performance by seeding the context window with a number of example tasks, along with solutions that demonstrate the desired output.

This format of providing few-shot examples which facilitate in-context learning is commonly accepted by LLM APIs natively in the form of a messaging API. Each message in OpenAI’s Chat Completions API is tagged as either a *system* message, which is used to provide general instructions or set the tone of the conversation, *user* message which contains the user query in the form of a question, or an *assistant* message, demonstrating the expected answer.

The final message in the sequence is commonly a user message containing the desired prompt that the LLM is supposed to answer in the style of preceding messages. Figure 2 illustrates the structure of our in-context learning prompt format when generating based on a stub called `claim_rewards` (a part of CTF-09), demonstrating the sequence of system message, few-shot examples, and the final user query.

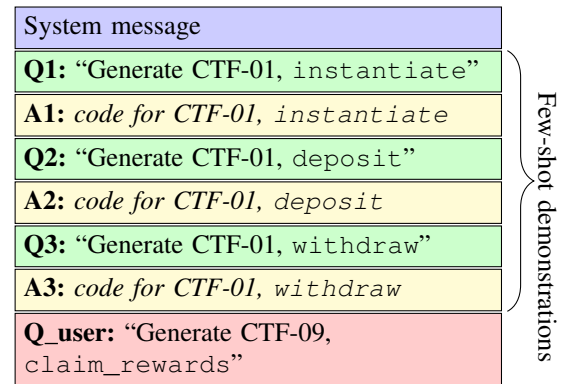


Fig. 2. Message format illustrating the use of few-shot examples for in-context learning.

³<https://github.com/informalsystems/FuzzMo/>

In our few-shot examples, each question emulates a user asking the model to generate a particular Quint pure function that encapsulates the logic of a message handler found in CTF-01 (instantiate, deposit, or withdraw). The answer, emulating the LLM’s response, is the correct implementation taken from our hand-crafted model for CTF-01. A partial template we use to create the question prompts is provided in Listing 3 (see Appendix E-A for the full version). We use the same template to format the final user prompt, containing the query for the target Quint function of the model we are currently generating.

As shown by our experimental evaluation, presented in part in Table I, this prompting style resulted in highly successful initial generations, frequently eliminating the need for further repair steps. We hypothesize that this high success rate is due to two key factors. First, the mechanical generation phase manages a significant portion of the task, reducing the potential for errors that would otherwise burden the LLM. Second, we enrich the LLM’s context window with highly relevant information derived from analyzing generated stubs and the Rust source code of the contract.

```
Please complete this stub for my '@@NAME@@@' Quint
function.
@@@DESCRIPTION@@@
'''
@@@STUB@@@
'''
Here is the contract state type and related types:
'''
@@@QUINT TYPE DEFINITIONS@@@
'''
<omitted>
The '@@NAME@@@' Quint function models a CosmWasm
Rust function for processing smart contract
messages.
@@@MESSAGE HANDLER@@@
Your Quint code must be functional.
All function arguments are immutable, and you can
not use mutable variables.
@@@IO EXAMPLES@@@
```

Listing 3. Partial user prompt template

In the template from Listing 3, the `@@@NAME@@@` macro expands to the stub (pure function) name, such as `withdraw`, as demonstrated in the third few-shot example for CTF-01 in Figure 2. The `@@@DESCRIPTION@@@` macro expands to the user-provided natural language description of the stub’s intended functionality, as illustrated in Listing 4 for the `withdraw` stub in CTF-01. We obtain the expansion for `@@@QUINT TYPE DEFINITIONS@@@` by analyzing the output of the mechanical generation phase. Starting from the function signature of the placeholder at hand, we recursively look up relevant type definitions in the stub model’s Quint source code.

Coupled with a Quint cheatsheet with a list of commonly used operations that we always include in the system message, this provides the LLM with useful and targeted knowledge to execute complex operations on data and implement the desired functionality for a particular stub.

In order to expand the `@@@MESSAGE HANDLER@@@` macro, we search the contract’s CosmWasm source code for a block of Rust code that is a function definition beginning with `pub fn <stub name>`. Our mechanical generation tool guarantees that this function (the message handler which induced the placeholder) exists in the Rust source files.

Finally, the `@@@IO EXAMPLES@@@` macro expands to the input-output specifications provided by the user. (In our evaluation, we supply the LLM with two input-output specifications.) The full version of this template can be seen in Listing 19 in Appendix E.

```
"Enable users to withdraw from valid lockups,
updating the state, or return an error if validation
fails."
```

Listing 4. User-provided description for CTF-01, withdraw stub

Adapter Generation for Model-based Testing: As explained in Section III-A, model-based testing involves executing traces, sequences of state transitions generated by the model, against the implementation. An adapter is a Rust test which accomplishes this, but writing one manually is a time-consuming task. In order to fully automate a model-based testing workflow, we generate an adapter by following a procedure similar to the one outlined throughout Section IV: a baseline test which executes the trace can be generated mechanically, but we use an LLM to synthesize the state comparison function, a part of the adapter which checks whether contract states correspond with state transitions in the model.

While a trivial state comparison function can be mechanically generated (e.g., one that always returns true), we leverage the LLM’s instruction-following capabilities to generate a function that performs a more thorough inspection of the smart contract’s state. If we detect errors in the generated adapter code, we trigger a repair loop similar to the one we outline for model generation in the next section. In Appendix A, we conduct a case study for the model-based testing workflow supported by adapter generation, and relegate the technical details to Appendix D.

C. Testing and Repair Phase

In this section, we outline our strategy for addressing potential errors that arise during generation. As highlighted in Section IV-B, we independently repair each function in the model. Because we never generate code with side effects, it suffices that the generated code correctly implements the intended behavior for all I/O specifications. This stems from the fact that the model’s overall structure and state management code have been generated deterministically. We divide errors into three distinct categories, based on the earliest point at which they can be detected, namely

- 1) **static errors** (e.g., incorrect Quint syntax), which are detected before executing any tests;
- 2) **runtime errors** (e.g., calling a nonexistent method), which are detected when the generated is executed; and
- 3) **semantic errors**, which are detected once the generated pure function returns an output that is inconsistent with a user-provided input-output specification.

After the initial generation is completed, we analyze it for the presence of either static, runtime, or semantic errors. If we detect them, our strategy is to enter a loop of test-and-repair rounds, as shown in Figure 1. In each round, we gather relevant information about the probable cause (such as error messages) and construct a prompt asking the LLM to repair the code based on this context. The LLM’s response then replaces the previous, erroneous fragment of code, and we check the new version for correctness again.

In order to prevent an unbounded number of LLM calls, we define a finite repair round “budget” for each error category: the amount of times the system may invoke an LLM to repair a particular type of error. In all of our experiments, we use a budget of 3 repair rounds for each error category, leading to a maximum of 9 additional LLM calls.

Although this approach does not guarantee an error-free result, in practice we have found it sufficiently capable of achieving our goal: reducing the developer time necessary to utilize Quint models. As shown in our experimental evaluation, in most runs we were able to successfully generate correct code for most target functions and contracts in our benchmark, using fewer than 3 rounds of repair on average. We outline the pseudocode for this phase (the TESTANDREPAIR function in Algorithm 1), in Algorithm 2 of Appendix B-A.

In the following two sections, we further explain the static and semantic repair rounds. When repairing runtime errors, we use a combination of methods and prompts employed for static and semantic errors (details in Appendix B-B).

Static Errors: To detect static errors, we utilize Quint’s type checking tool (which will, naturally, also detect syntax errors). An example of this tool’s output when detecting a syntax error is presented in Listing 5. This error occurred during code generation for a stub named `execute_add_strategy` in our real-world Neutron DAO case study.

Quint being a new language means that commercial LLMs lack comprehensive information on Quint-specifics. The error in Listing 5 arises from the new and limited pattern matching syntax in Quint. The generated code attempts to pattern-match on literal values, a valid pattern in many languages that support this feature. However, as the error message points out, this is not yet supported in Quint. (In this case, a parser-informed method such as PICARD [19] would prove valuable.)

Using our repair prompting methodology, the LLM is able to correctly reason about the root cause of the error, and subsequently produces correct Quint code. Although we lack a dataset of few-shot examples suitable for our repair phase, we compensate by instructing the LLM to start the solution with reasoning about the problem before writing any code [36]. For brevity, we omit the corrected code, and instead showcase the LLM’s reasoning in Listing 6. The template we use to generate static repair prompts can be found in Appendix E-B.

```
[...]quint/model_generated.qnt:86:20 - error: [
QNT000] mismatched input 'true' expecting {'_',
LOW_ID, CAP_ID}
86:      | Ok(true) => (Err("Cannot remove last
      | administrator"), state)
      | ^^^^^
```

```
[...]quint/model_generated.qnt:87:20 - error: [
QNT000] mismatched input 'false' expecting {'_',
LOW_ID, CAP_ID}
87:      | Ok(false) => (
      | ^^^^^
```

Listing 5. Static error in Neutron DAO

```
2. **Mismatched input 'true' expecting {'_', LOW_ID,
CAP_ID}**: This error indicates that the use of the
boolean literals 'true' and 'false' directly in
pattern matching is not allowed in Quint.
3. **Mismatched input 'false' expecting {'_', LOW_ID
, CAP_ID}**: Similar to the previous error, the use
of the boolean literal 'false' directly in pattern
matching is not allowed.
```

Listing 6. LLM reasoning

Semantic Errors: After embedding the synthesized function into the mechanically generated stub file and confirming it passes type checking, we verify whether it respects the user-provided I/O specifications.

Let F denote the true Quint pure function that implements a particular part of the contract’s logic in the model. In this setting, F is unknown, and we only assume access to $K \in \mathbb{N}$ user-provided I/O specifications ($K \geq 2$), and $\tilde{F}$, the LLM’s completion for a stub of F . Mechanical generation guarantees that F and $\tilde{F}$ share the same type signature (arguments and return type), and we require that $\tilde{F}$ must pass type checking before semantic repair rounds are triggered. Thus, we may call $\tilde{F}$ with arguments suitable for F , and if $\tilde{F}$ returns, the return value will be comparable with the return value of F .

Semantic repair rounds run only if there are no runtime errors. Each I/O specification is represented as a pair (args_i, R_i) ($1 \leq i \leq K$), where args_i denotes the arguments applied to F or $\tilde{F}$ (the user-specified input), and $R_i = F(\text{args}_i)$ denotes the expected return value (the user-specified output). We identify a semantic error if $\tilde{F}(\text{args}_i) \neq R_i$ for a given i . In that case, we start a semantic repair round by preparing a specialized prompt, employing a template similar to the one used for static errors.

We provide to the LLM (1) args_i , the user-specified input data; (2) $\tilde{F}(\text{args}_i)$, the incorrect output of the current implementation; and (3) R_i , the user-specified expected output. Of course, just as in a static repair round, the prompt also contains other information such as the erroneous implementation ($\tilde{F}$). In this way, we provide sufficient information to correct the function’s behavior at a number of points in the input space, and we rely on the LLM’s generalization capabilities to enhance the overall correctness.

V. EVALUATION

In Section V-A, we evaluate our Quint model generation technique on a benchmark consisting of 5 smart contracts from the CosmWasm Capture the Flag repository and present summary results. In Appendix C, we present more comprehensive results for the entire benchmark, and perform a limited ablation study investigating the effect of natural language descriptions on performance across the CTF Benchmark.

In Section V-B, we present a case study applying our method to a real-world audit scenario. An additional case study, capturing the full model-based testing workflow, can be found in Appendix A.

A. Benchmark: CosmWasm Capture the Flag

The CosmWasm Capture the Flag (CTF) repository provides the source code for 10 smart contracts used in a capture-the-flag competition. Each smart contract in the repository contains a critical bug, and the goal in a CTF competition is to find the bug and craft an exploit around it. For example, in CTF-01, users may deposit funds into a lockup, and withdraw funds from lockups by specifying a lockup ID. The bug can be found in the `withdraw` message handler, which does not check if the same lockup ID appears multiple times in the message, enabling users to withdraw all funds from the same lockup twice, and thus steal funds from the contract. We focused our benchmark on contracts from the CosmWasm CTF repository where the vulnerability is caused by a bug in program logic rather than by a technical or platform-dependent reason, making Quint models useful for detecting the issue.

Our benchmark consists of 23 target functions, drawn from our hand-crafted reference Quint models for CTFs 02, 04, 05, 07, and 09. We used the CTF-01 contract and its model for in-context learning, as described in Section IV-B: we crafted a series of Question-Answer pairs, each pair demonstrating how to fulfill the task of generating implementations (Answer) given Quint placeholders (Question).

Furthermore, we wrote a natural-language description of each target function, along with 5 I/O examples. We use 2 I/O examples to emulate a user's input, and the remaining 3 as a partial specification for post-generation inspection. (Each function was additionally manually inspected for correctness.)

In this benchmark, our tool generated and repaired a Quint model for each of the contracts. To manage stochasticity inherent in LLM outputs, we repeat this experiment across 5 independent runs. We made a best-effort attempt to maximize the reproducibility of our experiments. In addition to a frozen model checkpoint, we provide timestamps, system fingerprints, and the seeds used for all LLM API calls.

We present the results of our benchmark for CTF-04 and 09 in Table I, showing that the final, post-repair, results generate a function that satisfies the supplied I/O specification in almost all runs (indicated by the green checkmark in the first column). Full results can be found in Appendix C, in Table III.

We supply the first two I/O examples as part of the initial generation request, and to perform test-and-repair rounds. The column **Avg. Repair** indicates the average number of repair rounds (static, runtime, and semantic) that were used for generating each function. If the generated code contains no runtime errors, we evaluate the functions on the remaining three I/O examples, that the LLM did not have access to during generation, and report the results as the **Pass Rate**.

To provide a clearer view of each contract's difficulty, Table II details the total number of repair rounds performed per contract, categorized by type of repair.

TABLE I
AVERAGE NUMBER OF REPAIR ROUNDS USED PER FUNCTION AND PASS RATES FOR UNSEEN TEST CASES, AVERAGED ACROSS 5 INDEPENDENT RUNS FOR CTF-04 AND CTF-09

CTF-04			
	Function	Avg. Repair	Pass Rate
✓	instantiate	0.0	100%
✓	mint	1.2	90%
✓	burn	0.0	100%
CTF-09			
	Function	Avg. Repair	Pass Rate
✓	deposit	2.4	73%
? (4/5)	withdraw	1.6	73%
✓	increase_reward	0.2	100%
? (4/5)	claim_rewards	1.8	80%

TABLE II
DISTRIBUTION OF ERROR REPAIR ROUNDS BY TYPE

Contract	Static	Runtime	Semantic	Σ
CTF-02	0	0	3	3
CTF-04	1	5	0	6
CTF-05	0	0	0	0
CTF-07	4	0	0	4
CTF-09	23	0	7	30

More detailed results can be found in Appendix C. They follow the trends described in this section and provide one additional insight: the quality of the NL description degrades the performance only slightly, allowing the method to work well even if the user fails to provide a precise description.

B. Case Study: Privileged subDAO Contract

In this case study, we applied the described approach to a project from one of the audits that Informal performed recently. The scope of the audit was a new functionality of Neutron's [37] governance contracts. To provide sufficient context, let us describe the basic concepts of on-chain governance.

Most blockchains, including Neutron, typically do not have a single chain owner. Instead, the chain is jointly owned and governed by all the token holders. Any user may create a *governance proposal*, and the token holders vote for or against it. If there is sufficient support for the proposal, it gets automatically executed on the chain. We call such an organizational structure a *decentralized autonomous organization* (DAO).

The whole governance process, from proposal creation to its execution, takes a non-negligible amount of time. Furthermore, some proposals may be very technically specialized (e.g., regular adjustment of particular chain parameters) and thus not sufficiently interesting for members of the DAO. To address these issues, the newly added functionality introduced the concept of a *privileged subDAO*: a smaller set of users authorized to vote in and execute specific changes only (before execution, the decision can still be overridden by members of the main DAO).

The functionality is implemented in the `neutron-chain-manager` contract. Different subDAOs can either have an `AllowAll` authorization (typically,

for the main or security DAO, which can do anything) or an `AllowOnly([Permission])` authorization, which specifies a set of permissions that a subDAO can execute. The relevant entry point functions of the contract are `AddStrategy` (which gives an authorization to a subDAO), `RemoveStrategy` (which removes it), and `ExecuteMessages` (which executes a message if it has authorization to do so). Overall, the contract contains 16 functions that together implement the desired functionality.

The main property that needs to be preserved is the following: `ExecuteMessages` that attempts to execute a message for which the sender subDAO does not have permission will fail. Unfortunately, in the audited commit, the property was not satisfied. The problem was identified through manual code analysis and fixed for the release.

Identifying the bug using model generation: We took the code at the audit commit and wanted to see if we could find the same bug using model generation. We first applied the mechanical generation step described in Section IV-A, and then the model generation described in Section IV-B. The artifacts resulting from this case study are available on GitHub.⁴

Mechanical generation reliably produced stubs as expected. There was some additional cleanup work needed: removing references to external libraries or unused types, and removing stubs for contracts that were out of scope. Furthermore, some adjustment was needed for handling message passing of the custom `NeutronMsg` type. Overall, the effort to go from the generated version to a working model stub was neither trivial nor too big.

Out of 16 functions that there were in the contract, we identified 6 of them relevant for deciding on the correct authorization. For those 6 functions, we ran the model generation process.

Out of 6 functions, 4 were immediately generated correctly. The remaining two functions were fixed with syntactic repair. The problem was that at the first attempt, there were issues in calling functions within the generated code (once the name was hallucinated, and once the order of parameters did not match the signature).

One interesting observation compared to the reference model (a model written by the authors manually): at a couple of places we had to make sure that a property holds for all elements of a list. Thus, we factored that functionality out and wrote the helper function `listforall` (following the naming of the equivalent function on sets, `forall`). The generated model, since it always generates only one function at a time (without the possibility of creating helper functions), generated expressions that were more complicated compared to the reference implementation.

Finally, the generated Quint model sufficed to uncover the authorization bug that existed in the contract. This presents the main takeaway of the case study: the presented approach could indeed be used in a real-world audit.

Beside the main takeaway, there are a couple of peculiarities worth commenting on. First, we see that there was no need for semantic repair of generated model functions. This is because the authorization mostly depends on getting the types right, and the logic is fairly simple.

Second, because our model stub ensures full generality handling CosmWasm bank transfers, and the analyzed contracts did not include any, the whole model ended up less readable than a model an experienced Quint user would have written by hand. The tradeoff is then between ease of modeling and ease of reading. Because these models are used for bug finding, and not for protocol documentation (which is another use case for Quint models), we find that making modeling easier is the right choice for all but the most proficient Quint modelers. Contrary to the full generality of mechanically generated stubs, we found the LLM-generated parts of the models to be written in a very natural way.

There are two caveats regarding the optimistic takeaways of this section. First, our method presents the core around which an ergonomic, user-friendly tool should be built. Second, generalizing the method to smart contract environments other than CosmWasm would require separate design process for the stub generation part.

VI. CONCLUSION

In this paper, we reported on our effort to bridge the gap between model-based techniques and software auditing practice. We did so by automatically generating both Quint models and model-based tests, taking a CosmWasm contract, I/O examples, and a high-level description as input. We demonstrated how our technique is able to generate correct models on a capture-the-flag benchmark, and ran a successful case study on a past audit.

Our experience shows that LLMs can be tamed by using mechanical generation for setting up a firm frame for statistical code generation. Iterated repair attempts (syntactic, semantic, and runtime) were useful for correcting the LLM's occasional hallucinations. While the proposed method does not guarantee correctness, our evaluation shows that it works well in practice. Furthermore, at every step of the generation process (model stubs, models, test stubs, tests), the method provides useful artifacts for the users, which is very much appreciated by practitioners.

There are still aspects of our technique that need to be improved if we want to make it fully practical. Currently, the input-output examples we expect from users are encoded as Quint files with Quint syntax. Providing a better user experience there would go a long way to bring our technique to a broader audience. Additionally, mechanical generation, while fully reliable, is still restricted to CW contracts that do not send messages among themselves. This limits the applicability of the technique to simpler contracts.

The main practical benefit of our approach is easing auditors' way into using model-based techniques—a task with a very steep learning curve prior to having our tool available.

⁴https://github.com/informalsystems/fuzzmo_use_case2/

REFERENCES

- [1] A. C. Dias Neto, R. Subramanyan, M. Vieira, and G. H. Travassos, "A survey on model-based testing approaches: a systematic review," in *Proceedings of the 1st ACM international workshop on Empirical assessment of software engineering languages and technologies: held in conjunction with the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE)* 2007, 2007, pp. 31–36.
- [2] Cosmwasm book. [Online]. Available: <https://book.cosmwasm.com/>
- [3] Quint. [Online]. Available: <https://quint-lang.org/>
- [4] L. Lamport, "The temporal logic of actions," *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 3, pp. 872–923, 1994.
- [5] Cosmwasm capture the flag competition tasks. [Online]. Available: <https://github.com/oak-security/cosmwasm-ctf>
- [6] Jepsen. [Online]. Available: <https://jepsen.io>
- [7] L. Lamport, *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [8] Tla+ ibc specifications. [Online]. Available: <https://github.com/informalsystems/hermes/tree/master/docs/spec/tla>
- [9] Reference implementation of eth2.0 in dafny. [Online]. Available: <https://github.com/ConsensSys/eth2.0-dafny>
- [10] K. R. M. Leino, "Accessible software verification with dafny," *IEEE Softw.*, vol. 34, no. 6, pp. 94–97, 2017. [Online]. Available: <https://github.com/dafny-lang/dafny>
- [11] Certora prover. [Online]. Available: <https://www.certora.com/prover>
- [12] Slither code repository. [Online]. Available: <https://github.com/crytic/slither>
- [13] Mythril code repository. [Online]. Available: <https://github.com/ConsensSys/mythril>
- [14] A. Groce and G. Grieco, "echidna-parade: A tool for diverse multicore smart contract fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 658–661.
- [15] Kontrol code repository. [Online]. Available: <https://github.com/runtimeverification/kontrol>
- [16] Halmos code repository. [Online]. Available: <https://github.com/al6z/halmos>
- [17] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *CoRR*, vol. abs/2107.03374, 2021.
- [18] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. Canton-Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," *CoRR*, vol. abs/2308.12950, 2023.
- [19] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: parsing incrementally for constrained auto-regressive decoding from language models," *CoRR*, vol. abs/2109.05093, 2021. [Online]. Available: <https://arxiv.org/abs/2109.05093>
- [20] G. Poesia, A. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=KmtVD97J43e>
- [21] C. Sun, Y. Sheng, O. Padon, and C. Barrett, "Clover: Closed-loop verifiable code generation," 2024. [Online]. Available: <https://openreview.net/forum?id=oSuVEv4X7w>
- [22] Z. Sun, X. Du, Z. Yang, L. Li, and D. Lo, "Ai coders are among us: Rethinking programming language grammar towards efficient code generation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1124–1136.
- [23] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.
- [24] Y. Charalambous, N. Tihanyi, R. Jain, Y. Sun, M. A. Ferrag, and L. C. Cordeiro, "A new era in software security: Towards self-healing software via large language models and formal verification," *arXiv preprint arXiv:2305.14752*, 2023.
- [25] X. Ye, Q. Chen, I. Dillig, and G. Durrett, "Optimal neural program synthesis from multimodal specifications," *arXiv preprint arXiv:2010.01678*, 2020.
- [26] N. Jain, S. Vaidyanath, A. S. Iyer, N. Natarajan, S. Parthasarathy, S. K. Rajamani, and R. Sharma, "Jigsaw: Large language models meet program synthesis," in *ICSE*. ACM, 2022, pp. 1219–1231.
- [27] M. R. H. Misu, C. V. Lopes, I. Ma, and J. Noble, "Towards ai-assisted synthesis of verified dafny methods," *arXiv preprint arXiv:2402.00247*, 2024.
- [28] R. Wang, J. Zhang, Y. Jia, R. Pan, S. Diao, R. Pi, and T. Zhang, "Theoremllama: Transforming general-purpose llms into lean4 experts," *arXiv preprint arXiv:2407.03203*, 2024.
- [29] Y. Yu, P. Manolios, and L. Lamport, "Model checking tla+ specifications," in *CHARME*, ser. Lecture Notes in Computer Science, vol. 1703. Springer, 1999, pp. 54–66.
- [30] I. Konnov, J. Kukovec, and T. Tran, "TLA+ model checking made symbolic," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, pp. 123:1–123:30, 2019.
- [31] D. Jackson, *Software Abstractions: logic, language, and analysis*. MIT press, 2012.
- [32] J. Corazza, I. Gavran, G. Moreira, and D. Neider, "Accessible smart contracts verification: Synthesizing formal models with tamed llms," 2025. [Online]. Available: <https://arxiv.org/abs/2501.12972>
- [33] Rustc plugin. [Online]. Available: https://github.com/cognitive-engineering-lab/rustc_plugin
- [34] cw-multi-test cosmwasm testing framework. [Online]. Available: <https://github.com/CosmWasm/cw-multi-test>
- [35] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *CoRR*, vol. abs/2005.14165, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [36] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. H. Chi, Q. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models," *CoRR*, vol. abs/2201.11903, 2022. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [37] Neutron github org. [Online]. Available: <https://github.com/neutron-org>